

AADK 12: Task 1 — Advanced Theming

Strategy: Material 3 + Brand Identity

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 12

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Research Theming Concepts

colorScheme

colorScheme in Material 3 defines the set of colours used across the UI (primary, secondary, background, surface, etc.) to maintain visual consistency.

Example:

```
private val LightColorScheme = lightColorScheme(  
    primary = Color(0xFF0066CC),  
    secondary = Color(0xFF03DAC5),  
    tertiary = Color(0xFFFF9800)  
)
```

typography

typography defines the text styles used in the app such as headings, body text, labels, and captions.

Example:

```
val AppTypography = Typography(  
    headlineLarge = TextStyle(  
        fontSize = 30.sp,  
        fontWeight = FontWeight.Bold
```

```
),  
    bodyLarge = TextStyle(  
        fontSize = 16.sp  
    )  
)
```

shapes

shapes define the corner radius and geometric styling used for UI components such as buttons, cards, and dialogs.

Example:

```
val AppShapes = Shapes(  
    small = RoundedCornerShape(4.dp),  
    medium = RoundedCornerShape(12.dp),  
    large = RoundedCornerShape(24.dp)  
)
```

Dynamic Colour (Android 12+)

Dynamic colour allows the app theme to adapt automatically based on the user's wallpaper colours on Android 12+ devices.

Example:

```
val colorScheme = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {  
    dynamicLightColorScheme(context)  
} else {  
    LightColorScheme  
}
```

Light/Dark Theme Support

Light and dark themes adjust colours based on system appearance settings to improve readability and reduce eye strain.

Example:

```
val colors = if (isSystemInDarkTheme()) {  
    DarkColorScheme  
} else {  
    LightColorScheme  
}
```

2. Brand + Theme Design

App Brand

App Name: ZooTracker
Logo Idea: Minimal lion head icon with leaf pattern (symbolizing wildlife conservation).
Brand Style: Clean, modern, nature-inspired colours.

Colour Palette

Light Theme

Token	Hex Value
Primary	#2E7D32
Secondary	#0288D1
Tertiary	#FFB300
Background	#F5F5F5
Surface	#FFFFFF
Error	#B00020

Dark Theme

Token	Hex Value
Primary	#66BB6A
Secondary	#4FC3F7
Tertiary	#FFD54F
Background	#121212
Surface	#1E1E1E
Error	#CF6679

Typography Tokens

Token	Font Size	Font Family
Heading	28 sp	SansSerif
Subtitle	20 sp	SansSerif
Body	16 sp	SansSerif
Caption	12 sp	SansSerif

Example:

```
val Typography = Typography(  
    headlineLarge = TextStyle(fontSize = 28.sp),  
    titleMedium = TextStyle(fontSize = 20.sp),  
    bodyLarge = TextStyle(fontSize = 16.sp),  
    bodySmall = TextStyle(fontSize = 12.sp)  
)
```

Shape Tokens

Shape Token	Value	Usage
smallCorner	4 dp	Small buttons
mediumCorner	12 dp	Cards
largeCorner	24 dp	Dialogs / large surfaces

Example:

```
val Shapes = Shapes(  
    small = RoundedCornerShape(4.dp),  
    medium = RoundedCornerShape(12.dp),  
    large = RoundedCornerShape(24.dp)  
)
```

3. Theme Architecture Document

Theme File Structure

```
ui/theme/  
├── Color.kt  
├── Typography.kt  
├── Shapes.kt  
└── Theme.kt
```

Color.kt

Defines light and dark colour palettes.

Typography.kt

Defines text styles used throughout the app.

Shapes.kt

Defines corner radius tokens.

Theme.kt

Combines all theme elements and applies them to the app.

Applying Theme in Compose

@Composable

```
fun ZooTrackerTheme(content: @Composable () -> Unit) {  
    val colorScheme = if (isSystemInDarkTheme()) {  
        DarkColorScheme  
    } else {  
        LightColorScheme  
    }  
}
```

```
MaterialTheme(  
    colorScheme = colorScheme,  
    typography = Typography,  
    shapes = Shapes,  
    content = content  
)  
}
```

Wrapping the App UI

```
setContent {  
    ZooTrackerTheme {  
        MainScreen()  
    }  
}
```

Using Theme Tokens in Components

Instead of hard-coding values:

✗ Bad practice

```
Text(color = Color.Red)
```

✓ Good practice

```
Text(color = MaterialTheme.colorScheme.primary)
```

This ensures consistent styling across the entire app.

4. Theming Across UI Components

AppBar

Property	Theme Token
Colour	colorScheme.primary
Text Style	typography.titleLarge
Shape	shapes.medium

Example:

```
TopAppBar(  
    colors = TopAppBarDefaults.topAppBarColors(  
        containerColor = MaterialTheme.colorScheme.primary  
    )  
)
```

CardItem

Property	Theme Token
Background	colorScheme.surface
Typography	typography.bodyLarge

Property	Theme Token
Shape	shapes.medium
Elevation	4.dp

Example:

```
Card(
  shape = MaterialTheme.shapes.medium,
  colors = CardDefaults.cardColors(
    containerColor = MaterialTheme.colorScheme.surface
  )
)
```

Button

Property	Theme Token
Colour	colorScheme.secondary
Text Style	typography.bodyLarge
Shape	shapes.small
Elevation	2.dp

Example:

```
Button(
  shape = MaterialTheme.shapes.small,
  colors = ButtonDefaults.buttonColors(
    containerColor = MaterialTheme.colorScheme.secondary
  )
)
```

5. Reflection

A well-structured theming system ensures consistency across all UI components and makes it easier to update the app design in one place. If colours and styles are hard-coded in multiple components, it becomes difficult to maintain and update the design later.

Using centralized theme tokens improves scalability and helps teams maintain design standards. Dynamic colour enhances personalization by adapting the app's theme to the user's wallpaper, while light and dark themes improve accessibility and reduce eye strain in different lighting conditions. This ultimately creates a more modern and user-friendly experience.